

Intro

lunedì 4 maggio 2026 16:23

Il **NIST** è un'agenzia che definisce la **sicurezza dei computer** come:

misure e controlli che garantiscono la **confidenzialità, integrità e disponibilità (CIA)** delle informazioni di un sistema (HW, SW, firmware, informazioni processate, salvate e comunicate)

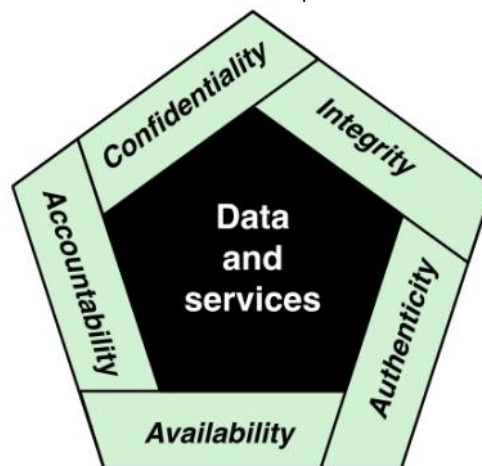
Questi tre concetti sono riferiti come **triade CIA**:

- **Confidenzialità (Riservatezza)**: preservare le restrizioni autorizzate sull'accesso e la divulgazione delle informazioni, garantendo metodi per proteggere la privacy personale e le informazioni proprietarie.
Una **perdita di confidenzialità** è la divulgazione non autorizzata di informazioni
- **Integrità**: proteggersi contro modifiche/eliminazioni delle informazioni, garantendone la non ripudiabilità e l'autenticità
Una **perdita di integrità** è la modifica/eliminazione non autorizzata di informazioni
- **Disponibilità**: assicurare l'accesso e uso tempestivo e affidabile delle informazioni.
Una **perdita di disponibilità** è l'interruzione all'accesso o uso delle informazioni di un sistema informativo

Spesso viene **estesa (CIA+)** per rispondere meglio alle minacce moderne:

- **Autenticità**: La proprietà di:
 - Essere genuino, verificabile e affidabile
 - Fiducia nella validità di trasmissione di un messaggio o del suo mittenteQuindi poter verificare che gli utenti siano chi dichiarano di essere e che ogni input nel sistema arrivi da una fonte affidabile
- **Responsabilità**: l'obiettivo che impone che le azioni di un'entità siano tracciabili unicamente a quell'entità. Questo supporta la:
 - Non ripudiabilità
 - Deterrenza
 - Isolamento dai guasti
 - Rilevamento e prevenzione da intrusioni
 - Recupero di eventi e azioni legali

Bisogna essere in grado di tracciare una violazione di sicurezza alla parte responsabile. I sistemi devono mantenere registrazioni delle loro attività per permettere alle successive analisi forensi di rintracciare le violazioni di sicurezza o per aiutare in controversie sulle transazioni



I **livelli di impatto** servono per quantificare la gravità di una violazione di sicurezza.

Lo standard FIPS 199 definisce tre livelli di impatto sulle operazioni/asset/individui di un'organizzazione:

- **Basso**: perdita con **effetto avverso limitato**. L'organizzazione può ancora svolgere le funzioni primarie nonostante la ridotta efficacia
Es: divulgazione di dati sugli esiti degli esami (confidenzialità), modifica di un testo in una pagina web (integrità), temporanea indisponibilità di un app online per la ricerca di una directory telefonica (disponibilità)
- **Moderato**: perdita ha un **effetto avverso serio**. Causa un degrado significativo nella capacità di compiere la missione dell'organizzazione causando danni significativi agli asset e agli individui (senza causare morti o lesioni)
Es: divulgazione di dati di iscrizione degli studenti (confidenzialità), perdita di integrità di un forum online, indisponibilità del sito web universitario (disponibilità)
- **Alto**: perdita con **effetto catastrofico/severo**. L'organizzazione perde la capacità di eseguire una o più delle sue funzioni principali con danni gravissimi finanziari e agli individui (anche con rischio di morte)
Es: compromissione di un account di root (confidenzialità), alterazioni di prescrizioni mediche in un database (integrità), blocco di servizi di autenticazione per sistemi critici (disponibilità)

Capire la Sicurezza Informatica

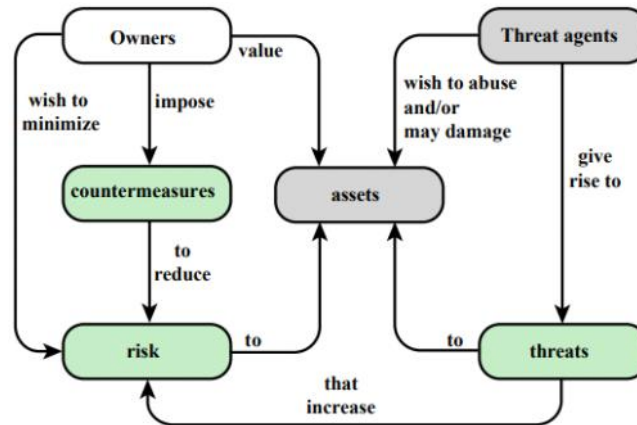
La sicurezza informatica è un campo complesso e per progettare e mantenere un sistema sicuro bisogna superare diverse sfide fondamentali:

- 1) **Complessità non evidente**: la sicurezza non è semplice come appare ad un novizio.
- 2) **Anticipare l'avversario**: nello sviluppo di un meccanismo di sicurezza bisogna considerare potenziali attacchi su questi meccanismi
- 3) **Procedure controintuitive**: spesso le contromisure sembrano inutilmente elaborate ma perfettamente logiche quando si comprende la natura della minaccia
- 4) **Posizionamento logico e fisico**: bisogna decidere dove implementare i meccanismi di sicurezza sia fisicamente nella rete sia logicamente nel software
- 5) **Gestione dei segreti**: molti meccanismi si basano sul possesso di informazioni segrete (es chiavi crittografiche) e la loro creazione, distribuzione e protezione è un problema complesso
- 6) **Asimmetria tra attaccante e difensore**: gli attaccanti devono trovare una sola falla di sicurezza mentre il difensore deve eliminarle tutte per una sicurezza completa
- 7) **Sicurezza come parte del sistema**: spesso la sicurezza viene implementata solo dopo che il design è completato invece di essere implementata come parte del design
- 8) **Monitoraggio continuo**: la sicurezza richiede un monitoraggio continuo per difendersi da minacce sempre più aggiornate
- 9) **Percezione del valore**: utenti e manager tendono a non percepire il beneficio dell'investire nella sicurezza finché non si verifica una violazione

10) **Sicurezza vs Usabilità**: spesso si ritiene che una forte sicurezza sia un'ostacolo alla facilità d'uso per gli utenti

Un modello per la Sicurezza Informatica

- **Risorse di sistema (Asset)**: entità di valore per l'organizzazione. Si divide in HW, SW, Dati (file, DB, ecc) e reti (collegamenti locali, router, ecc)
- **Vulnerabilità (di un Asset)**: debolezza nel sistema informativo, nelle procedure, nei controlli interni o nell'implementazione che può essere sfruttata. Generalmente un sistema può diventare **corrotto** (integrità), **leaky o permeabile** (confidenzialità) o **indisponibile** (disponibilità)
- **Minaccia**: qualsiasi circostanza o evento con il potenziale di causare danni alle operazioni/individui/asset organizzative o alla nazione, tramite accesso non autorizzato, divulgazione, distruzione, modifica dei dati o negazione del servizio (DoS)
- **Threat Agent (Attaccante)**: un individuo, gruppo, organizzazione o governo che conduce (o ha l'intento di condurre) attività dannose sfruttando le vulnerabilità
- **Attacco**: minaccia che tenta di raccogliere, interrompere, negare, degradare o distruggere risorse del sistema informativo
- **Contromisura**: dispositivo o tecnica usata per prevenire, rilevare o recuperare dagli effetti di un attacco, riducendo la vulnerabilità
- **Rischio**: misura di **quanto un'entità sia minacciata**. È tipicamente una funzione sulla **probabilità che un evento si verifichi (P)** e l'**impatto che ne deriverebbe (D)**. L'obiettivo è ridurre il rischio ad un livello accettabile
 $R = P \times D$
- **Security Policy**: insieme di regole e criteri che definiscono e vincolano le attività di una struttura di elaborazione dati al fine di mantenere una condizione di sicurezza accettabile



Minacce e Attacchi

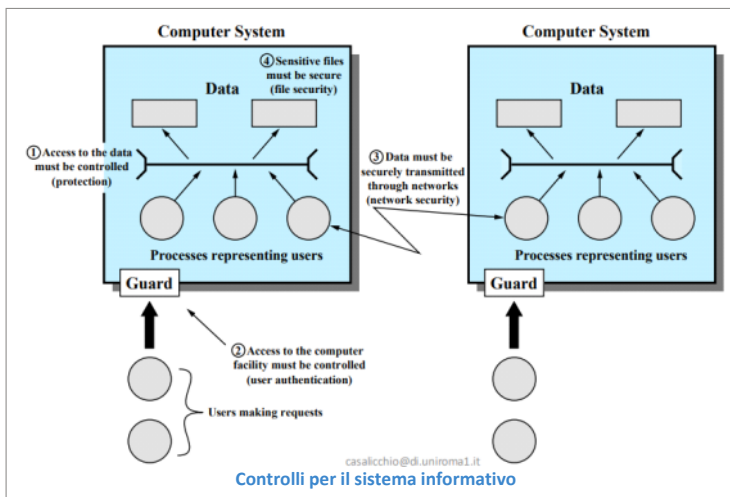
mercoledì 6 maggio 2026 16:14

Come abbiamo detto:

- **Vulnerabilità:** debolezza del sistema informativo che si divide in:
 - **Corrotto** (perdita di integrità)
 - **Leaky o Permeabile** (perdita di confidenzialità)
 - **Indisponibile** (perdita di disponibilità)
- **Minaccia:** capace di sfruttare vulnerabilità e rappresenta un danno potenziale agli asset
- **Attacchi:** minaccia eseguita, si divide in:
 - **Passiva:** tentativo di imparare o usare informazioni dal sistema che non influenzano le risorse
 - **Attiva:** tentativo di alterare le risorse del sistema o le sue operazioni
 - **Insider:** iniziato da un'entità all'interno del perimetro di sicurezza
 - **Outsider:** iniziato dall'esterno del perimetro

Quando una minaccia si concretizza in un attacco, può produrre diverse conseguenze negative. Secondo lo standard RFC 4949 possiamo classificarle in:

Conseguenze	Azione (Attacco)
Divulgazione non autorizzata (Unauthorized Disclosure) Un'entità ottiene l'accesso a dati per i quali non è autorizzata. Violazione della confidenzialità	Esposizione: dati sensibili rilasciati direttamente a un'entità non autorizzata (per errore o intenzionalmente) Intercettazione: un'entità non autorizzata accede ai dati mentre viaggiano tra la fonte e il destinatario Inferenza: accesso indiretto ai dati . L'entità deduce l'informazione analizzando le caratteristiche della comunicazione (es analisi del traffico) pur non accedendo necessariamente all'informazione vera e propria Intrusione: aggiornamento attivo delle protezioni di sicurezza di un sistema per rubare o esporre informazioni
Inganno (Deception) Un'entità autorizzata riceve dati falsi credendoli per veri	Impersonificazione: un'entità non autorizzata ottiene l'accesso o esegue un attacco fingendosi un'entità autorizzata Falsificazione: dati falsi ingannano un'entità autorizzata Ripudio: un'entità inganna un'altra negando falsamente le proprie responsabilità
Interruzione (Disruption) Interrompe o previene il corretto funzionamento dei servizi di sistema	Incapacitazione: disabilitazione di un componente di sistema (es malware che blocca l'esecuzione di certi servizi) Corruzione: alterazione indesiderata delle funzioni di sistema Ostruzione: azione che interrompe la consegna di servizi ostacolando le operazioni di sistema
Usurpazione (Usurpation) Controllo non autorizzato del servizio o delle funzioni di sistema	Appropriazione indebita: un'entità assume il controllo logico o fisico delle risorse di sistema Uso improprio: Uso delle funzionalità di sistema in modo dannoso per la sua stessa sicurezza



Controlli per il sistema informativo

	Availability	Confidentiality	Integrity
Hardware	Equipment is stolen or disabled, thus denying service.	An unencrypted CD-ROM or DVD is stolen.	
Software	Programs are deleted, denying access to users.	An unauthorized copy of software is made.	A working program is modified, either to cause it to fail during execution or to cause it to do some unintended task.
Data	Files are deleted, denying access to users.	An unauthorized read of data is performed. An analysis of statistical data reveals underlying data.	Existing files are modified or new files are fabricated.
Communication Lines and Networks	Messages are destroyed or deleted. Communication lines or networks are rendered unavailable.	Messages are read. The traffic pattern of messages is observed.	Messages are modified, delayed, reordered, or duplicated. False messages are fabricated.

Esempi di minacce

Classificazione degli Attacchi

Gli attacchi si dividono in due famiglie in base all'impatto che hanno sul sistema:

Attacchi Passivi

Tentativi di apprendere o usare informazioni del sistema **senza alterare le risorse di sistema**. L'attaccante monitora le trasmissioni con l'obiettivo di ottenere le informazioni trasmesse. La difesa punta **alla prevenzione** (es crittografando i dati) più che al rilevamento.

Si divide in:

- **Rilascio del contenuto dei messaggi:** intercettazione del contenuto del messaggio
- **Analisi del traffico:** anche con un messaggio cifrato un attaccante può studiare l'intestazione, locazione degli host e dedurre frequenza e dimensione dei dati scambiati

Attacchi Attivi

Tentativi di **alterare le risorse o le operazioni di sistema**, modificando o falsificando il flusso dei dati. Sono raggruppati in quattro categorie:

- **Replay:** cattura passiva di un dato e la sua successiva ritrasmissione per produrre un effetto non autorizzato
- **Impersonificazione:** assunzione illecita di un'altra identità, solitamente accompagnata da altre forme di attacchi attivi
- **Modifica dei messaggi:** un messaggio viene ritardato, riordinato o alterato per cambiare l'ordine delle istruzioni
- **Denial of Service (DoS):** prevenzione e soppressione delle comunicazioni o sovraccarico che blocca le normali operazioni di rete

Requisiti di Sicurezza

Lo Standard FIPS 200 stabilisce un approccio per categorizzare le contromisure necessarie a proteggere la triade CIA.

Definisce **17 aree di contromisure** classificandole in base alla loro natura. Le aree si dividono in tre categorie principali:

- **Contromisure Tecniche:** controlli eseguiti dall'OS
 - 1) **Controllo degli Accessi (Access Control):** limita l'accesso alle risorse
 - 2) **Identificazione e Autenticazione (Identification and Authentication):** verifica l'identità di un'entità

- 3) **Protezione dei sistemi e delle comunicazioni (Systems and Communication Protection)**: garantisce la confidenzialità e l'integrità dei dati in transito
- 4) **Integrità del sistema e delle informazioni (System and Information Integrity)**: identifica alterazioni non autorizzate a livello di file o di memoria
- **Contromisure Funzionali**: gestione della sicurezza e del fattore umano, disciplinando il comportamento degli utenti e dell'organizzazione
 - 5) **Consapevolezza e Formazione (Awareness and Training)**: istruendo il personale si mitigano vulnerabilità legate all'errore umano
 - 6) **Controllo e Responsabilizzazione (Audit and Accountability)**: generazione, revisione e protezione dei log di sistema per poter ricostruire a posteriori le azioni di specifici utenti, garantendo la non ripudiabilità.
 - 7) **Certificazione, Accredimento e Valutazione di Sicurezza (Certification, Accreditation and Security Assessment)**: controllo periodico per vedere se i meccanismi di sicurezza funzionano
 - 8) **Pianificazione dell'emergenza (Contingency Planning)**: piani di backup e ripristino dati nei casi di guasti o attacchi
 - 9) **Manutenzione (Maintenance)**: manutenzione attiva del sistema informativo (es controllo se utenti hanno le autorizzazioni necessarie)
 - 10) **Protezione fisica e ambientale dei server (Physical and Environmental Protection)**: limita l'accesso fisico ai server e protegge da minacce fisiche (es incendi)
 - 11) **Pianificazione (Planning)**: sviluppo e aggiornamento dei piani di sicurezza e le regole dei comportamenti degli utenti che accedono alle risorse
 - 12) **Sicurezza del Personale (Personnel Security)**: background check prima dell'assunzione e policy di revoca tempestiva degli accessi al momento del licenziamento.
 - 13) **Valutazione dei Rischi (Risk Assessment)**: identificazione proattiva delle minacce e calcolo dell'impatto potenziale prima che si verifichino incidenti
 - 14) **Acquisizione di sistemi e servizi (Systems and Service Acquisition)**: garantisce che la sicurezza sia integrata nel sistema tramite servizi di terze parti (es antivirus)
- **Insieme di Contromisure Tecniche + Funzionali**: intersezione tra policy gestionali e strumenti tecnologici
 - 15) **Gestione della configurazione (Configuration management)**:
 - **Aspetto Tecnico**: software automatizzati devono forzare queste configurazioni e rilevare deviazioni illecite
 - **Aspetto Funzionale**: l'organizzazione deve documentare gli standard sicuri per tutti i dispositivi
 - 16) **Risposta agli incidenti (Incident Response)**:
 - **Aspetto Tecnico**: uso di sistemi di rilevamento delle intrusioni (IDS) e piattaforme per individuare, isolare la minaccia e raccogliere prove forensi
 - **Aspetto Funzionale**: esistenza di un piano per stabilire chi chiamare, come gestire le pubbliche relazioni e obblighi legali in caso di data breach
 - 17) **Protezione dei media (Media Protection)**:
 - **Aspetto Tecnico**: meccanismi crittografici implementati su dispositivi di archiviazione mobile e cancellazione sicura prima dello smaltimento
 - **Aspetto Funzionale**: regole per come conservare supporti di memoria (es chiavette USB)

Principi Fondamentali di Progettazione della Sicurezza

L'ingegneria della sicurezza si affida ad un set di principi guida, inizialmente formalizzati da Saltzer e Schroeder, venendopoi integrati con ulteriori concetti.

- **Economia del Meccanismo**: il design delle misure di sicurezza, sia HW che SW, deve essere **più semplice e piccolo possibile**. Così è facile da testare e verificare e facile da aggiornare e rimpiazzare
- **Fail-safe Defaults**: le decisioni di **accesso** devono **basarsi** sul **permesso** e non sull'esclusione. Il comportamento di default deve essere di **negare l'accesso** e lo schema di protezione definisce le condizioni in cui l'accesso è permesso.
- **Mediazione completa**: ogni accesso ad una risorsa deve essere **controllato dal meccanismo di accesso** e questo sistema deve essere inaggrabile e sempre invocato. I sistemi non dovrebbero basarsi su decisioni di accesso memorizzate in cache
Es se un sistema legge i permessi solo all'apertura di un file e i permessi dell'utente vengono revocati mentre il file è aperto, la modifica non avrà effetto immediato
- **Open Design**: l'architettura dei meccanismi di sicurezza **non deve essere segreta**. Questo consente ad uno scrutinio più ampio e indipendente
- **Separazione dei Privilegi**: richiede che per accedere ad una risorsa bisogna **soddisfare molteplici condizioni o attributi di privilegio** (es autenticazione a più fattori), in modo tale da aggiungere più difese.
Si applica anche al codice, dividendolo in parti con accessi limitati agli specifici privilegi necessari per una operazione, mitigando i danni in caso di compromissione del processo di base
- **Privilegio Minimo**: ogni processo e ogni utente deve operare usando il **set minimo di privilegi** strettamente necessari per il proprio compito. Questo limita il raggio d'azione di un incidente o di un attacco.
Es: su Linux vanno assunti i privilegi di root solo per il tempo limitato all'azione critica (sudo cmd) e poi subito abbandonati per evitare danni accidentali
- **Minimo Meccanismo Comune**: il design deve **ridurre al minimo** le funzioni di sistema, percorsi HW o SW che sono condivisi tra utenti o processi diversi. Riduce il numero di percorsi di comunicazione non voluti e la quantità di HW e SW su cui gli utenti
- **Accettabilità psicologica**: i meccanismi di sicurezza **non devono interferire** eccessivamente col lavoro degli utenti
- **Isolamento**: si applica in tre contesti:
 - I **sistemi di accesso pubblico** devono essere isolati dalle risorse
 - I **processi e file** degli utenti devono essere isolati l'uno dall'altro nella memoria (tranne quando è esplicitamente desiderato)
 - I **meccanismi di sicurezza** devono essere isolati dal sistema operativo per evitarne la manomissione
- **Incapsulamento**: affine alla programmazione orientata agli oggetti; protegge un'entità informatica celando le strutture dati e permettendo l'interazione solo attraverso interfacce predefinite e controllate
- **Modularità**: richiede lo sviluppo di funzioni di sicurezza in **moduli protetti e indipendenti** (es funzioni di crittografia) da usare in una **architettura modulare** per facilitare l'aggiornamento, testing o sostituzione con nuove funzionalità di sicurezza senza dover riprogettare l'intero OS
- **Layering**: uso di **molteplici approcci di protezione sovrapposti e ridondanti** che coprono persone, tecnologie e operazioni (es Firewall + IDS + Antivirus + Formazione). Fornisce barriere sequenziali tra l'attaccante e l'obiettivo e il fallimento o l'aggiramento di un singolo controllo non lascia il sistema indifeso
- **Stupore Minimo**: il programma o l'interfaccia utente dovrebbe sempre rispondere nel modo esatto in cui l'utente si aspetta. Poiché interfacce ambigue portano l'utente a confermare inconsapevolmente azioni pericolose

Superfici di Attacco e Alberi di Attacco

La **superficie di attacco** consiste nelle vulnerabilità raggiungibili e sfruttabili in un sistema (es porte di rete aperte, form web, servizi disponibili nel firewall, ecc) Si dividono in tre categorie:

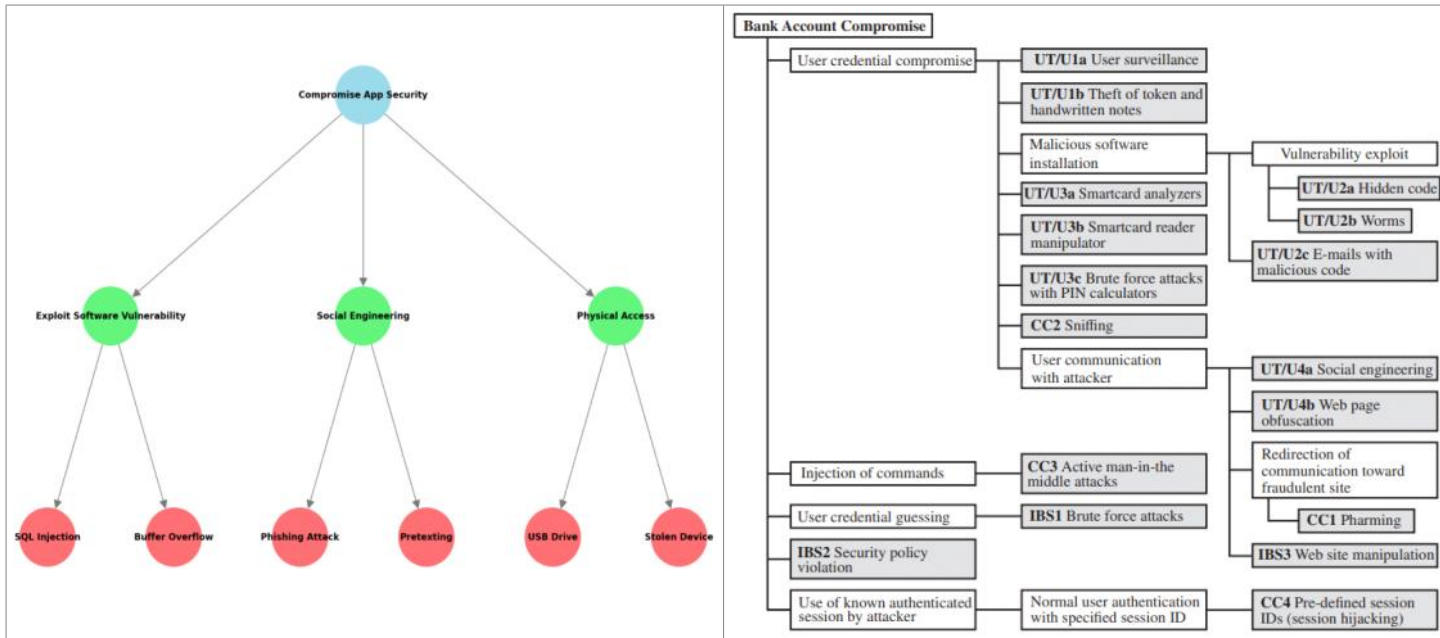
- **Superfici di Attacco su Rete**: protocolli di rete vulnerabili (es quelli usati negli attacchi DoS), interruzione di collegamenti di comunicazione e varie forme di attacchi da parte di intrusi
- **Superfici di Attacco su Software**: vulnerabilità nelle applicazioni, servizi o codice dell'OS. Focus particolare va dato al software dei server web
- **Superfici di Attacco su Umani**: vulnerabilità create da personale o esterni, tipo l'ingegneria sociale, errore umano e fonti interne fidate

Alberi di Attacco

Un **albero di attacco (Attack Trees)** è una struttura dati gerarchica e ramificata che modella l'insieme delle tecniche per sfruttare le vulnerabilità di sicurezza Il modello si divide in diversi livelli:

- **Radice**: obiettivo finale dell'attacco o il principale incidente di sicurezza da analizzare (es compromettere un conto bancario online)

- **Nodi intermedi:** sotto-obiettivi necessari per arrivare alla radice. Sono legati da logica:
 - **AND:** tutti i sotto-obiettivi devono essere raggiunti per passare al livello superiore
 - **OR:** raggiungere almeno uno dei sotto-obiettivi ci basta per procedere
- **Nodi foglia:** diversi modi per iniziare l'attacco (es crack della password WiFi)



Strategie della Sicurezza Informatica

Progettare un ambiente sicuro richiede una visione d'insieme e non solo la pura tecnica. Una strategia completa si basa su tre aspetti fondamentali:

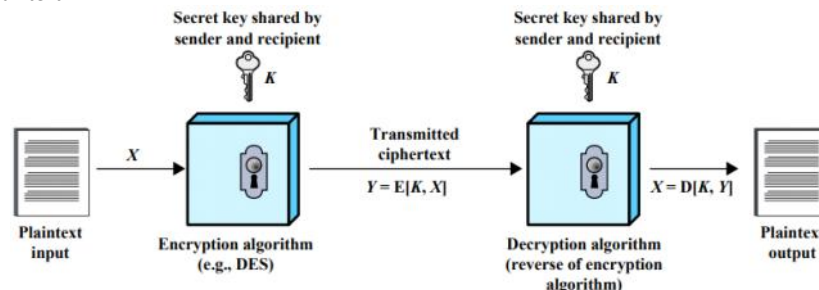
- **Specifica/Policy:** cosa deve fare lo schema di sicurezza? qual è l'obiettivo della protezione?
La policy è il primo passo concettuale. Come minimo deve definire informalmente i comportamenti desiderati del sistema (requisiti di sicurezza, integrità e disponibilità). La sua forma ideale è una **dichiarazione informale di regole e pratiche** che specificano o vincolano come un'organizzazione o un sistema fornisce servizi di sicurezza per proteggere gli asset sensibili.
Nello sviluppo bisogna considerare il valore degli asset, vulnerabilità e attacchi e due compromessi necessari:
 - **Facilità d'uso Vs Sicurezza:** le misure di sicurezza introducono ostacoli (es password complesse) e una sicurezza eccessiva può rendere inutilizzabile il sistema
 - **Costo della sicurezza Vs Costo del fallimento e del recupero:** i meccanismi di protezione hanno costi monetari e prestazionali che vanno bilanciati in funzione del rischio e del potenziale danno nel caso di attacco
- **Implementazione della Sicurezza:** implementazione delle regole definite dalla policy
 - **Prevenzione (Prevention):** impedire che l'attacco abbia accesso (es crittografare i dati per renderli inaccessibili)
 - **Rilevamento (Detection):** sistemi di intrusione nel caso di attacco
 - **Risposta (Response):** se il sistema rileva un attacco in corso (es DoS) deve poter reagire per interrompere l'azione e limitare i danni
 - **Recupero (Recovery):** capacità di ripristinare il corretto funzionamento (es copie di backup affidabili)
- **Garanzia e Valutazione (Assurance and Evaluation):** controllo dell'efficacia delle difese
 - **Garanzia:** grado di fiducia sul fatto che le misure di sicurezza (tecniche e operative) funzionino come previsto.
Risponde alle domande "Il design del sistema rispetta i requisiti?" e "L'implementazione del sistema rispetta le sue specifiche?"
La garanzia quindi è basata sul grado di fiducia e non in termini di prova concreta se il design o l'implementazione sia corretta in se
 - **Valutazione:** processo che esamina un prodotto informatico rispetto a **determinati criteri**. Coinvolge testing e tecniche analitiche e formali per confrontare obiettivamente il grado di sicurezza

Crittografia Simmetrica

giovedì 7 maggio 2026 12:26

La **crittografia simmetrica**, nota anche come crittografia convenzionale o a chiave singola (single-key encryption), è la tecnica universale per fornire confidenzialità ai dati archiviati o in transito. Esso si basa su cinque ingredienti principali:

- **Plaintext (testo in chiaro)**: il dato originale fornito nell'input dell'algoritmo
- **Encryption algorithm (algoritmo di cifratura)**: algoritmo matematico che esegue varie trasformazioni logiche sul plaintext
- **Secret key (chiave segreta)**: fornita in input all'algoritmo, determina le esatte sostituzioni e trasformazioni da eseguire sul plaintext
- **Ciphertext (testo cifrato)**: messaggio reso incomprensibile prodotto come output. Dipende univocamente dal plaintext e dalla chiave segreta. Per un dato messaggio, l'uso di due chiavi diverse produrrà due testi cifrati differenti
- **Decryption algorithm (algoritmo di decifratura)**: algoritmo di cifratura eseguito in modo inverso. Prende il testo cifrato e la stessa identica chiave segreta usata in origine per riprodurre il plaintext



L'uso della crittografia simmetrica richiede il soddisfacimento di **due requisiti**:

- **Algoritmo di cifratura forte**: un avversario che conosce l'algoritmo e ha accesso a uno o più testi cifrati non deve essere in grado di decifrarli o ricavarne la chiave. L'algoritmo deve resistere anche se l'avversario entra in possesso di svariate coppie di plaintext e relativo ciphertext
- **Distribuzione sicura della chiave**: il mittente e il destinatario devono poter scambiare e ottenere copie della chiave in **modo totalmente sicuro** e mantenerla protetta da accessi non autorizzati. Altrimenti tutte le comunicazioni saranno immediatamente compromesse

Per recuperare il plaintext da un ciphertext gli attaccanti fanno uso di **due tipologie generali di attacco**:

- **Cryptanalysis (attacco crittoanalitico)**: attacco logico che fa affidamento sulla comprensione della natura dell'algoritmo, combinata a volte con la conoscenza delle caratteristiche generali del plaintext o di coppie plaintext-ciphertext. Sfrutta queste caratteristiche logiche per dedurre matematicamente la chiave utilizzata
- **Brute-Force Attack (attacco a forza bruta)**: si **testa ogni singola chiave possibile** finché non ottiene una traduzione intelligibile del ciphertext. In media bisogna testare almeno la metà di tutte le chiavi possibili

La crittografia simmetrica è divisa in tre dimensioni indipendenti

Operazione di Trasformazione	Numero di Chiavi Usate	Processamento del Plaintext
I tipi di operazione per trasformare il plaintext in ciphertext: • Sostituzione: ogni elemento del plaintext viene mappato e sostituito da un altro elemento • Trasposizione: gli elementi del plaintext non vengono cambiati nel valore ma riorganizzati e spostati di posizione Vincolo assoluto: nessuna informazione deve andare persa I sistemi reali più sicuri coinvolgono molteplici stadi incrociati di sostituzione e trasposizione	Se mittente e destinatario usano: • La stessa chiave: crittografia simmetrica, single-key o secret-key • Chiavi differenti: crittografia asimmetrica, two-key o public-key	Il modo in cui i dati plaintext vengono processati dall'algoritmo: • Block cipher (cifrario a blocchi): elabora i dati prendendo un blocco di elementi alla volta (es blocchi da 128 bit), producendo un blocco di output della stessa dimensione • Stream cipher (cifrario a flusso): elabora il flusso di dati in modo continuo, cifrando e producendo un singolo elemento alla volta (es bit per bit)

Algoritmi di Crittografia a Blocchi Simmetrici

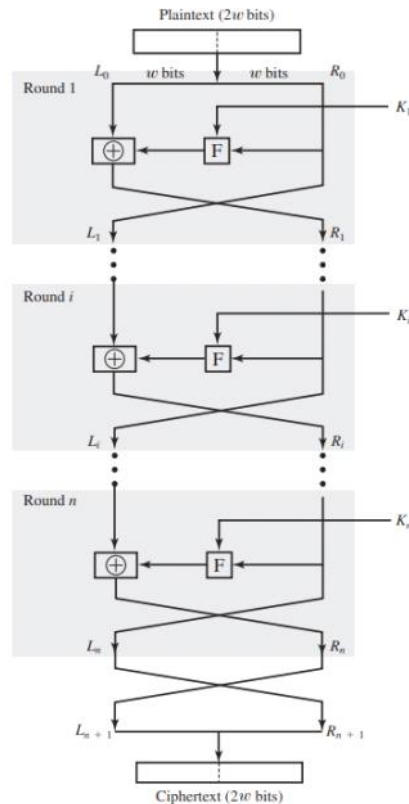
Gli algoritmi di crittografia simmetrica più usati sono **cifrari a blocchi**. Un cifrario a blocchi processa il plaintext in **blocchi di dimensione fissa** e produce blocchi di ciphertext di dimensione uguale per ogni blocco del plaintext.

Gli algoritmi simmetrici più importanti sono **Data Encryption Standard (DES)**, **triple DES (3DES)** e **Advanced Encryption Standard (AES)**

Architettura di un Block Cipher e Struttura di Feistel

La maggior parte dei cifrari a blocchi simmetrici usa la **struttura di Feistel**, che si articola su una precisa sequenza di iterazioni chiamate **round (giri)** che eseguono sostituzioni e permutazioni condizionate da una chiave segreta. Funzionamento del processo:

- 1) **Suddivisione dell'input**: il plaintext di lunghezza $2w$ bit viene **diviso in due metà identiche**, L_0 e R_0 , di lunghezza w bit
- 2) **Dinamica del round**: ogni round i riceve in input le metà precedenti L_{i-1} e R_{i-1} un **subkey** (K_i) differente, generata a partire dalla chiave principale K tramite un algoritmo di generazione
- 3) **Sostituzione e XOR**: nel round viene eseguita una sostituzione sulla metà sinistra: si applica una funzione matematica (**Round Function F**) alla metà destra parametrizzata dalla subkey K_i . Il risultato viene poi combinato tramite **XOR** con la metà sinistra
- 4) **Permutazione**: al termine della sostituzione avviene una permutazione che consiste nello **scambio di posizione delle due metà**
- 5) **Output**: dopo n round, le **due metà vengono ricombinate** per produrre il blocco di testo cifrato



Parametri di Progettazione e Sicurezza

La sicurezza e l'efficienza di un cifrario di Feistel dipendono dalla calibrazione di questi parametri:

- **Dimensione dei blocchi:** blocchi più grandi offrono maggiore sicurezza ma riducono la velocità di elaborazione. Lo standard è 128 bit
- **Dimensione della chiave:** chiavi più lunghe garantiscono una maggiore sicurezza ma riducono la velocità di elaborazione. Lo standard è 128 bit
- **Numero di round:** un solo round non offre sicurezza adeguata e l'aumento di round rende l'algoritmo esponenzialmente più robusto. Lo standard è 16 round
- **Algoritmo di generazione della subkey:** maggiore è la complessità dell'algoritmo di generazione delle subkey e maggiore sarà la difficoltà di crittoanalisi
- **Round Function (F):** una funzione F più complessa aumenta la resistenza ai tentativi di forzare il cifrario

Oltre alla pura sicurezza, ci sono altri due fattori critici:

- **Efficienza crittazione/decrittazione software:** spesso la cifratura è integrata in app SW senza HW dedicato, quindi la velocità di esecuzione diventa una preoccupazione
- **Facilità di analisi:** un algoritmo spiegato in modo chiaro e conciso è più semplice da analizzare per la ricerca di possibili vulnerabilità

Data Encryption Standard (DES)

Fino a poco fa, lo schema di crittografia più usato era basato sul DES, denominato **Data Encryption Algorithm (DEA)**.

Questo algoritmo è una variante della **rete di Feistel** e usa **blocchi** di plaintext da **64 bit** e una **chiave da 56 bit** e produce un blocco ciphertext da 64 bit.

Il processo prevede **16 round** di elaborazione e dalla chiave originale da 56 bit vengono generate 16 subkey, una per round.

9La decifratura è uguale alla cifratura ma le subkey vengono usate in **ordine inverso** (da K₁₆ a K₁)

Nonostante non si siano ancora trovate debolezze nell'algoritmo, il problema principale del DES ricade sulla **lunghezza della chiave**: con una chiave da 56 bit ci sono 2⁵⁶ chiavi possibili (≈ 7.2*10¹⁶). Un attacco brute-force con computer moderni richiederebbe pochissimo tempo per provare tutte le chiavi.

Triple DES (3DES)

L'uso di DES è stato esteso tramite l'uso di **Triple DES**, cioè **ripetendo** l'algoritmo **DES tre volte** usando due o tre chiavi univoche, portando la **lunghezza della chiave a 112 o 168 bit**.

Allungando la chiave si ha superato il problema dell'attacco brute-force e essendo l'algoritmo DES soggetto a molto scrutinio negli anni, rendendo ottimo l'uso di 3DES nella crittografia.

L'unico problema è che, essendo DES progettato per gli HW del 1970, l'algoritmo è un po' lento e, poiché 3DES esegue 3 volte i calcoli di DES, esso è ancora più lento.

Un altro lato negativo è che sia DES che 3DES usano blocchi da 64 bit

Quindi per motivi di sicurezza e efficienza, blocchi di grandezza maggiore sono necessari.

La sequenza di cifratura è:

- 1) **Cifratura** con chiave K₁
- 2) **Decifratura** con chiave K₂
- 3) **Cifratura** con chiave K₃ (che usando due chiavi sarà K₁ = K₃)

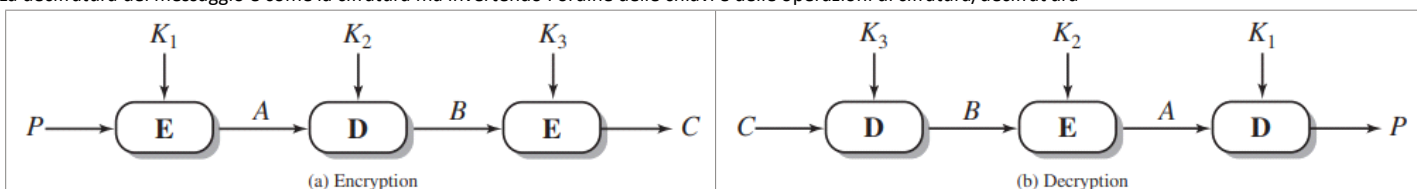
Formula:

$$C = E(K_3, D(K_2, E(K_1, P)))$$

La decifratura nel secondo stadio serve per la **compatibilità retroattiva** poiché se si imposta K₁ = K₂ = K₃ il risultato equivale a una singola cifratura DES standard

$$C = E(K_1, D(K_1, E(K_1, P))) = E(K, P)$$

La decifratura del messaggio è come la cifratura ma invertendo l'ordine delle chiavi e delle operazioni di cifratura/decifratura



Advanced Encryption Standard (AES)

L'**AES** è un cifrario a blocchi, ideato per rimpiazzare 3DES. NIST nel 1997 richiese di proporre questo nuovo standard in modo tale che abbia una sicurezza uguale o meglio di 3DES, più efficiente, e con cifrario a blocchi simmetrico con **blocchi di dati da 128 bit** e supporto di **chiavi da 128/192/256 bit**.

Rispetto ai predecessori che usavano l'architettura di Feistel, in cui il blocco di dati viene diviso a metà e elaborato in modo alternato, AES **processa l'intero blocco di dati in parallelo** durante ogni round.

Le caratteristiche principali sono:

- **Dimensione del blocco:** fissa a **128 bit**
- **Matrice State:** il blocco di input viene interpretato come una matrice quadrata 4x4 di byte (16 byte in totale). Tutte le operazioni di cifratura e decifratura vengono eseguite su questo "**State array**" (chiamato anche **in[i]**) che viene modificato a ogni stadio. Alla fine viene copiato su una matrice di output
- **Gestione delle chiavi:** AES supporta chiavi da 128/192/256 bit. La chiave originale viene espansa in un **Key Schedule** (array di word da 32 bit o 4 byte). Es per la chiave da 128 bit vengono generate 44 word. L'array di word Key Schedule viene chiamato per semplicità **w[i]**
- **Ordinamento:** l'ordinamento dei byte nella matrice avviene per **colonna**: i primi 4 byte dell'input occupano la prima colonna, i successivi 4 la seconda, ecc. Quindi i primi 4 byte dei 128 bit del plaintext vanno nella prima colonna della matrice **in** e i primi 4 byte (una word) della chiave espansa vanno nella matrice **w**

	DES	Triple DES	AES
Plaintext block size (bits)	64	64	128
Ciphertext block size (bits)	64	64	128
Key size (bits)	56	112 or 168	128, 192, or 256

Confronto dei tre algoritmi

Key Size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 decryptions/ μ s	Time Required at 10^{13} decryptions/ μ s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	$2^{55} \mu s = 1.125$ years	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	$2^{127} \mu s = 5.3 \times 10^{21}$ years	5.3×10^{17} years
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	$2^{167} \mu s = 5.8 \times 10^{33}$ years	5.8×10^{29} years
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	$2^{191} \mu s = 9.8 \times 10^{40}$ years	9.8×10^{36} years
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	$2^{255} \mu s = 1.8 \times 10^{60}$ years	1.8×10^{56} years

Tempo medio richiesto per la ricerca esaustiva della chiave

Quattro Fasi del Round

L'elaborazione si articola in **quattro fasi (stages)** distinte, una di permutazione e tre di sostituzione:

- 1) **Substitute Bytes (S-Box o substitution box):** sostituzione non lineare byte per byte tramite una tabella fissa. Distrugge le correlazioni matematiche semplici
- 2) **ShiftRows:** permutazione che sposta circolarmente le righe dello State. È fondamentale per "espandere" i byte di una colonna su quattro colonne diverse nel round successivo
- 3) **MixColumns:** trasformazione che altera ogni byte di una colonna in funzione di tutti gli altri byte della stessa colonna, garantendo diffusione dei bit
- 4) **AddRoundKey:** esegue un'operazione di XOR **bitwise** (bit a bit) tra la matrice State e la porzione corrente della chiave espansa

Queste fasi vengono ripetute più volte nella cifratura/decifratura: inizia con **1 round** di **AddRoundKey**, poi **9 round** (tutte quattro le fasi) e infine un **10° round** con solo tre fasi (S-Box, ShiftRows e AddRoundKey), così da rendere il cifrario reversibile.

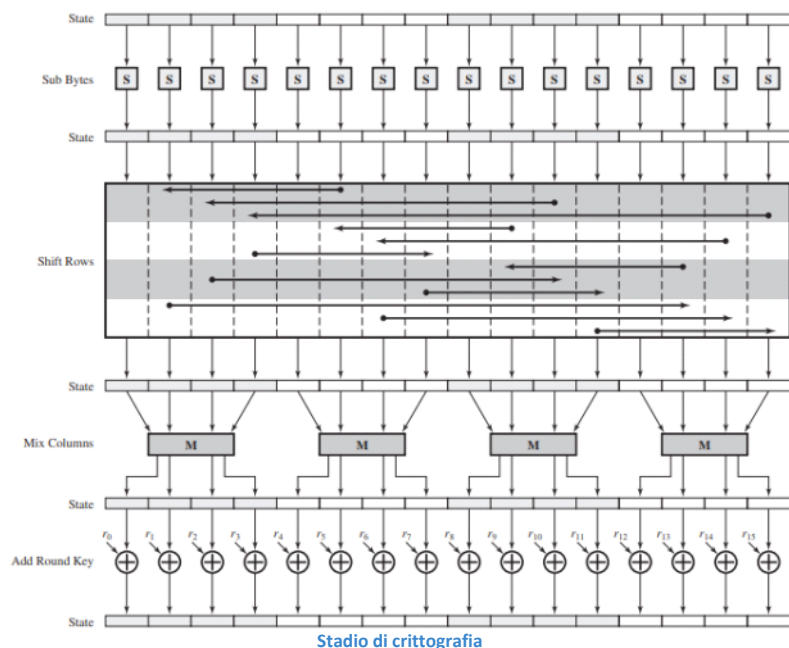
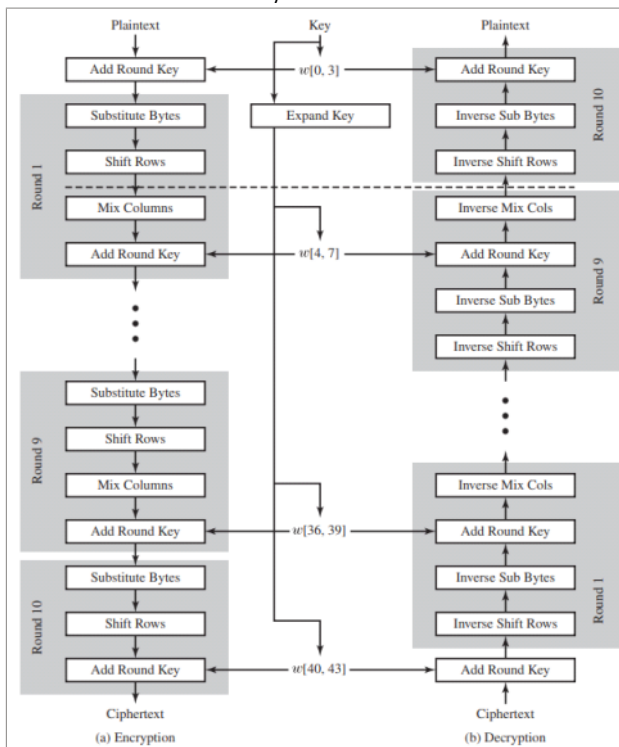
Questa struttura riflette una strategia di sicurezza definita "**alternanza di operazioni**". Lo stadio AddRoundKey da solo non è robusto e le altre tre fasi da sole (essendo reversibili e senza chiave) non offrirebbero protezione.

La sicurezza nasce dal concatenare operazioni di XOR con operazioni di rimescolamento (scrambling) rendendo il sistema estremamente resistente alla crittoanalisi.

Decifratura e Reversibilità

Ogni stadio dell'AES è facilmente reversibile, tuttavia, poiché non è una struttura di Feistel, l'algoritmo di **decifratura non è identico a quello di cifratura**:

- Vengono usate versioni inverse delle fasi (Inverse S-Box, Inverse Shift Rows, Inverse Mix Cols)
- Per la fase AddRoundKey l'inverso è ottenuto facendo lo XOR della stessa chiave sul blocco, usando il risultato che $A \oplus A \oplus B = B$

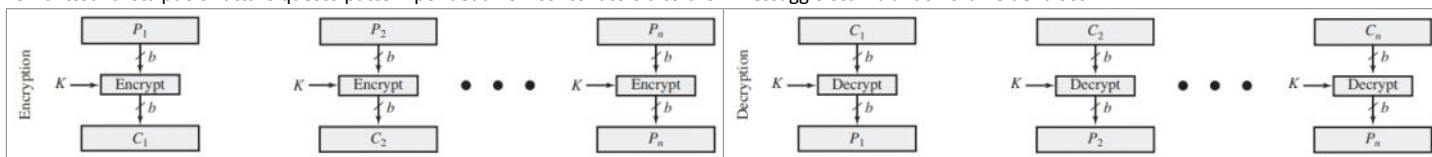


Problematiche Pratiche di Sicurezza e Electronic CodeBook (ECB)

Normalmente la crittografia simmetrica è applicata su dati più grandi di un solo blocco da 64 o 128 bit. I messaggi plaintext (es mail, pacchetti di rete, ecc) devono essere divisi in più blocchi di lunghezza fissa prima di essere cifrati.

Il sistema più semplice è l'**ECB** dove il messaggio viene frammentato in blocchi e ogni blocco viene **cifrato in modo indipendente**, usando la **stessa chiave**.

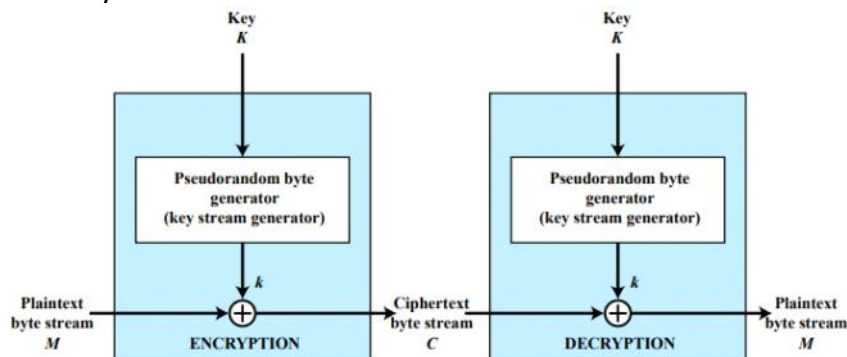
Per **messaggi lunghi** la modalità ECB è **insicura**: se lo stesso blocco plaintext appare più volte nel testo esso produrrà ogni volta lo stesso identico blocco cifrato. Un crittoanalista può sfruttare questo pattern per dedurre il contenuto o alterare il messaggio scambiando l'ordine dei blocchi



Cifrari a Flusso (Stream Ciphers)

A differenza dei cifrari a blocchi che elaborano un blocco di bit alla volta, i **cifrari a flusso** elaborano i dati in input in **modo continuo**, producendo un elemento (tipicamente un byte) alla volta. Il vantaggio principale è che sono più veloci e usano meno codice.

L'architettura tipica si basa su un **generatore pseudocasuale di Byte** che prende in input la chiave segreta e produce un flusso di numeri apparentemente casuali ma deterministicamente calcolabili, chiamato **keystream**.



Vedremo poi più in dettaglio i funzionamenti dei vari [algoritmi di crittografia](#)

Autenticazione dei Messaggi

venerdì 8 maggio 2026 11:53

La crittografia protegge da attacchi passivi (intercettazione), ma un sistema sicuro deve sapersi difendere anche da **attacchi attivi** (es falsificazione o manipolazione dei dati in transito).

Un messaggio/file si dice **"autentico"** quando proviene esattamente dalla fonte presunta. L'**autenticazione dei messaggi** è quindi la procedura che permette alle parti comunicanti di verificare tre aspetti cruciali:

- **Integrità**: i contenuti del messaggio non sono stati alterati
- **Autenticità della fonte (Source Authentication)**: la fonte è davvero quella dichiarata
- **Sequenza e Tempestività (Timeliness/Sequence)**: il messaggio non è stato ritardato o replicato abusivamente (replay attack)

Sebbene si possa direttamente crittografare il messaggio per garantire l'autenticazione a chi ha le chiavi, questo approccio a volte non è sempre conveniente/possibile. Infatti un messaggio può essere:

- **Con confidenzialità**: messaggio autenticato e crittografato o il messaggio e il suo tag di autenticazione sono crittografati
- **Senza confidenzialità**: messaggio autenticato ma non crittografato e un tag di autenticazione è prodotto e aggiunto al messaggio

Ci sono numerose situazioni in cui si preferisce autenticare un messaggio **senza cifrarlo** (lasciandolo plaintext):

- **Architetture broadcast**: il messaggio deve essere inviato a più destinazioni quindi è molto più economico inviare il messaggio in chiaro con un tag di auth
- **Carico di rete pesante**: se un nodo è sottoposto a carico eccessivo, non può permettersi il tempo di decifrare tutti i messaggi in ingresso
- **Programmi informatici**: l'autenticazione di un codice in plaintext è utilissima, poiché il programma non deve essere decifrato ogni volta al suo avvio, ma il suo tag di auth può essere controllato periodicamente.

Message Authentication Code (MAC)

Una tecnica molto diffusa per l'autenticazione si basa sull'uso di un **MAC**, un meccanismo che richiede che mittente (A) e destinatario (B) condividano una chiave segreta K_{AB} .

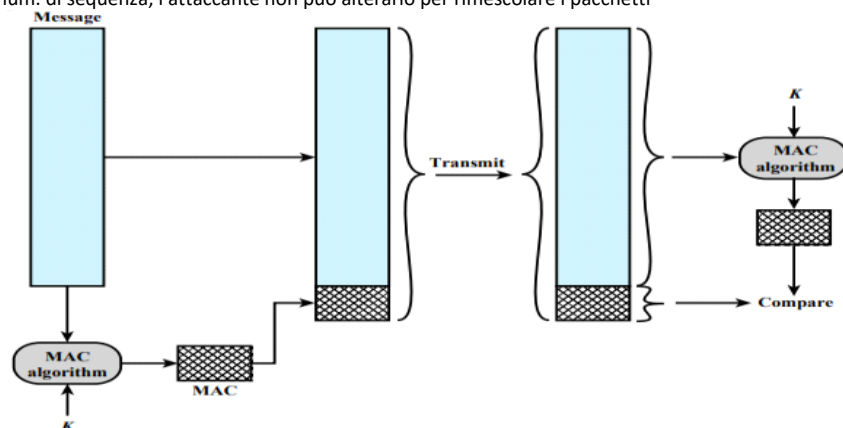
Quando A invia un messaggio M, applica un algoritmo MAC che genera un piccolo blocco di dati a lunghezza fissa, calcolato con una funzione sul messaggio e sulla chiave segreta

$$MAC = F(K_{AB}, M)$$

A trasmette il messaggio plaintext e il codice MAC e quando B lo riceve, ricalcola il MAC con la stessa chiave sul messaggio ricevuto e controlla se coincide con quello inviato da A.

Il destinatario ottiene garanzia assoluta (se il MAC coincide):

- Il messaggio **non è stato alterato** (l'attaccante se non ha la stessa chiave segreta non può ricalcolare un nuovo MAC valido per il messaggio modificato)
- Il messaggio **proviene dal mittente vero** (solo lui può aver generato il MAC corretto)
- Se il messaggio include un num. di sequenza, l'attaccante non può alterarlo per rimescolare i pacchetti



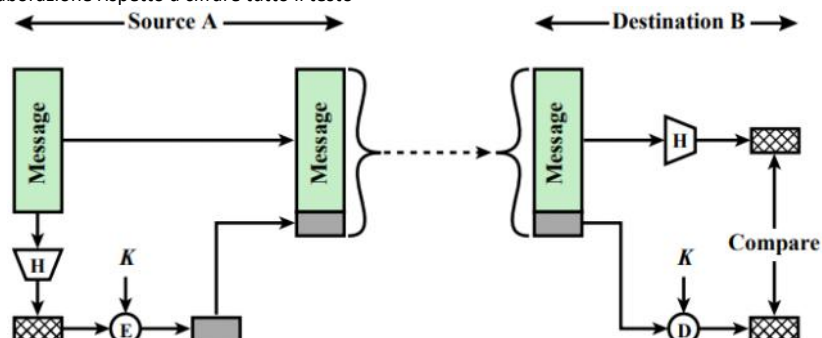
Funzioni di Hash Crittografiche (One-Way Hash Function)

Un'alternativa al MAC è l'uso di funzioni Hash che, a differenza del MAC, **non prendono una chiave segreta in input**, ma accetta un messaggio M di dimensione variabile in input e produce un messaggio di dimensione fissa $H(M)$ chiamato **Message Digest** (o codice di hash).

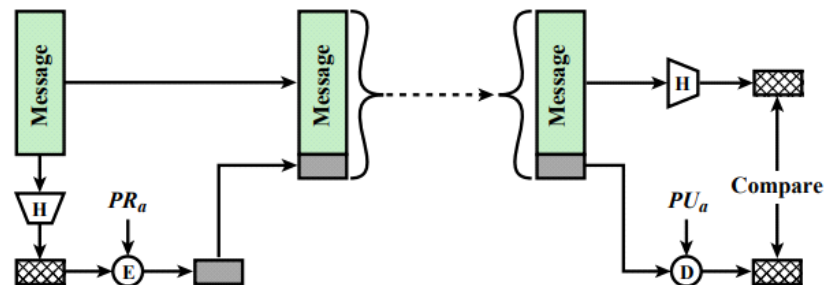
Per motivi di sicurezza il messaggio originale viene allungato (**padded**) a un multiplo di lunghezza fissa (es 1024 bit) includendo nel padding un campo contenente la lunghezza in bit del messaggio originale, rendendo estremamente difficile forgiare un messaggio fasullo con lo stesso hash.

Ci sono tre approcci principali per **"hashare" un messaggio**:

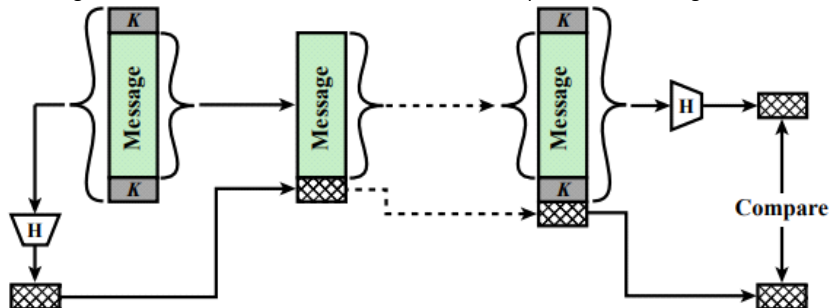
- **Cifratura simmetrica del Digest**: si calcola l'hash del messaggio plaintext e poi si cifra solo il piccolo codice di hash (usando la chiave segreta). Riduce drasticamente il tempo di elaborazione rispetto a cifrare tutto il testo



- **Cifratura a chiave pubblica del Digest (Firma Digitale)**: si cifra il codice di hash usando la chiave privata del mittente. Fornisce autenticazione e firma digitale senza necessitare di uno scambio preventivo di chiavi tra le parti



- **Uso del "Secret Value" (Keyed Hash):** metodo più rapido in assoluto perché non usa alcun algoritmo di cifratura. Si concatena la chiave segreta K col messaggio M e di nuovo con la chiave K (quindi si crea un blocco K-M-K). Si calcola l'hash dell'intero blocco $MD_M = H(K \parallel M \parallel K)$. Il mittente invia il messaggio M col digest MD_M . Così l'attaccante, non conoscendo K, non può ricalcolare il digest



Proprietà di Sicurezza di una Funzione Hash (H)

Per essere considerata crittograficamente sicura, una funzione hash deve soddisfare rigorosamente sei requisiti:

- 1) **Dimensione variabile:** H può essere applicata a un blocco di dati di una qualunque grandezza
- 2) **Output fisso:** H produce un output a lunghezza fissa
- 3) **Efficienza:** H(x) è computazionalmente facile da calcolare per facilitarne l'implementazione software e hardware
- 4) **Resistenza alla Pre-Immagine (One-Way):** dato un codice hash h, deve essere computazionalmente **infattibile ricavare il messaggio originale x** t.c. $H(x) = h$. Se fosse possibile invertire la funzione (cioè $x = H^{-1}(h)$) un attaccante potrebbe intercettare il messaggio M e il codice hash $MD_M = H(K \parallel M \parallel K)$, invertirla e trovare la chiave segreta K
- 5) **Resistenza alla Seconda Pre-Immagine (Weak Collision Resistance):** dato uno specifico blocco x, è computazionalmente **infattibile trovare un altro messaggio y** ($x \neq y$) t.c. **generino lo stesso hash** ($H(x) = H(y)$). Previene attacchi in cui l'attaccante intercetta il pacchetto e crea un messaggio falso che restituisca l'esatto hash criptato o originale
- 6) **Forte Resistenza alle Collisioni (Strong Collision Resistance):** è computazionalmente **infattibile trovare** una qualsiasi **coppia casuale di messaggi** (x, y) che **collidano sullo stesso hash** ($H(x) = H(y)$). Se questa regola non viene rispettata un truffatore può creare due contratti (uno da 10 e uno da 100000 euro) che producono lo stesso hash, farne firmare digitalmente il primo alla vittima e usare tale firma per incassare il secondo contratto

Se una funzione soddisfa le proprietà da 1 a 5 è detta **Weak Hash Function**, se rispetta anche la 6 è detta **Strong Hash Function**

Sicurezza e Standard: SHA e HMAC

La violazione degli hash si tenta solo tramite brute-force o crittoanalisi. Quindi la sicurezza contro brute-force dipende dalla lunghezza in bit del codice di hash. Per trovare collisioni (Strong Collision) la resistenza matematica è legata al limite di $2^{n/2}$.

Gli standard a livello mondiale appartengono alla famiglia **SHA (Secure Hash Algorithm)** emessa dal NIST:

- **SHA-1:** produce un hash da 160 bit. A causa di vulnerabilità strutturali oggi è considerata compromessa
- **SHA-2:** comprende varianti da 256, 384, 512 bit, attualmente considerate robuste
- **SHA-3:** nuovo standard con una struttura ideata per evitare i difetti di progettazione

Le funzioni hash sono anche usate per:

- Salvare le password in modo sicuro: invece di salvare la password in plaintext si salva il suo hash e al nuovo accesso si controlla se l'hash della password inserita coincide con quello salvato
- Controllare se un file è stato manomesso: si salva l'hash di ogni file e nel caso in cui uno venga manomesso genererà un hash differente da quello salvato

Crittografia Asimmetrica, RSA e Firma Digitale

venerdì 8 maggio 2026 14:43

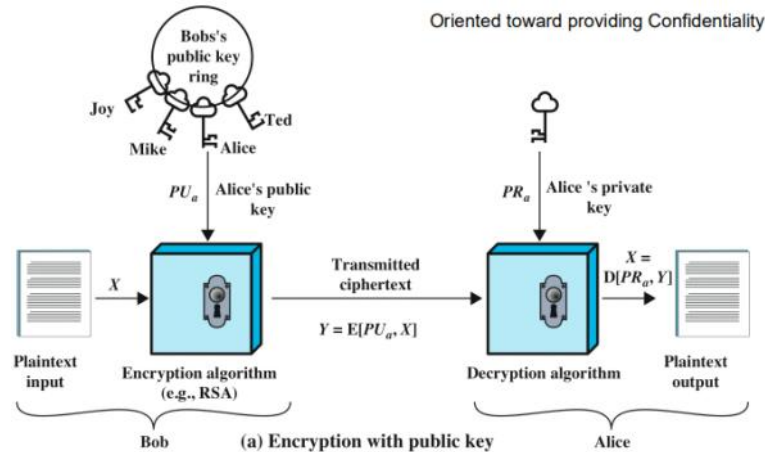
Crittografia Asimmetrica (a Chiave Pubblica)

La crittografia a **chiave pubblica**, proposta per la prima volta nel 1976, rappresenta il primo progresso rivoluzionario nella crittografia.

Gli algoritmi a chiave pubblica si basano su complesse **funzioni matematiche** e la caratteristica principale (**l'asimmetria**) risiede nell'uso di due chiavi correlate:

- **Public Key (Chiave Pubblica)**: resa pubblica e accessibile a chiunque per essere usata
- **Private Key (Chiave Privata)**: conosciuta e mantenuta segreta dal proprietario

I sistemi a chiave pubblica vengono principalmente applicati a: la firma digitale (autenticazione), distribuzioni di chiavi simmetriche e la cifratura per la confidenzialità



Gli algoritmi principali di crittografia asimmetrica sono:

- **RSA (Rivest, Shamir, Adleman)**, 1977: l'algoritmo più usato e implementato nella crittografia a chiave pubblica
Cifrario a blocchi dove il plaintext e ciphertext sono interi da 0 a $n-1$ per un certo n
- **Diffie-Hellman key exchange algorithm**, 1976: permette a due utenti di generare un segreto condiviso che può essere usato come chiave segreta nella successiva crittografia simmetrica dei messaggi. Limita lo scambio di chiavi
- **Digital Signature Standard (DSS)**: fornisce solo una funzione per la firma digitale con SHA-1 e non può essere usato per crittografia o scambio di chiavi
- **Elliptic Curve Cryptography (ECC)**: sicuro come RSA ma con chiavi più piccole

Algorithm	Digital Signature	Symmetric Key Distribution	Encryption of Secret Keys
RSA	Yes	Yes	Yes
Diffie-Hellman	No	Yes	No
DSS	Yes	No	No
Elliptic Curve	Yes	Yes	Yes

Uso del crittosistema a chiave pubblica

Firma Digitale

sabato 9 maggio 2026 18:03

Lo standard NIST FIPS PUB 186-4 definisce la **firma digitale** come: "una trasformazione crittografica di dati che, quando è implementata correttamente, fornisce un meccanismo di autenticazione dell'origine, integrità dei dati e non-ripudio da parte del firmatario". In questo modo, una firma digitale è una sequenza di bit dipendente dai dati, generata da un rappresentante come funzione di un file, messaggio o altro tipo di dato. FIPS 186-4 specifica l'uso di questi tre algoritmi per la firma digitale:

- RSA Digital Signature Algorithm
- DSA (Digital Signature Algorithm)
- Elliptic Curve Digital Signature Algorithm (ECDSA)

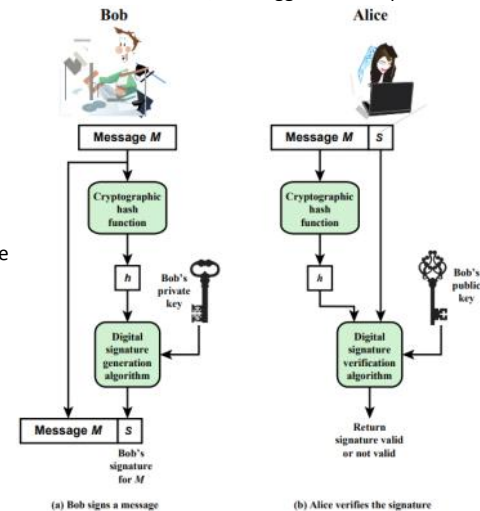
Es di funzionamento:

- 1) Il mittente (Bob) **genera l'hash** del messaggio M plaintext
- 2) Bob **cifra l'hash** usando la propria **chiave privata**. Il risultato è la firma digitale, che viene aggiunta al messaggio M
- 3) Il destinatario (Alice) riceve il messaggio, ne ricalcola l'hash e usa la **chiave pubblica** di Bob per decifrare la firma digitale ricevuta
- 4) Se l'hash calcolato su M da Alice coincide con quello decifrato con la chiave allora la firma è valida

Quindi fornisce una autenticazione della fonte e sull'integrità del messaggio ma non sulla confidenzialità

Vulnerabilità e Certificati a Chiave Pubblica

Lo schema della firma digitale crolla se un attaccante riesce a ingannare Alice spacciando la propria chiave pubblica per quella di Bob. Se ciò accade, l'attaccante può forgiare messaggi falsi e firmarli con la propria chiave privata e Alice usando la stessa chiave pubblica falsificata, verificherà con successo la firma credendo erroneamente che provenga da Bob. In questo modo Alice invierà i messaggi intesi per Bob all'attaccante.



La soluzione universale sono i **certificati a chiave pubblica (public-key certificates)**.

Un certificato è un pacchetto di dati che lega l'identità di un utente (User ID) alla sua chiave pubblica, garantito e **firmato digitalmente da una terza parte fidata** chiamata **Certificate Authority (CA)** (es agenzia governativa, InfoCert, ecc).

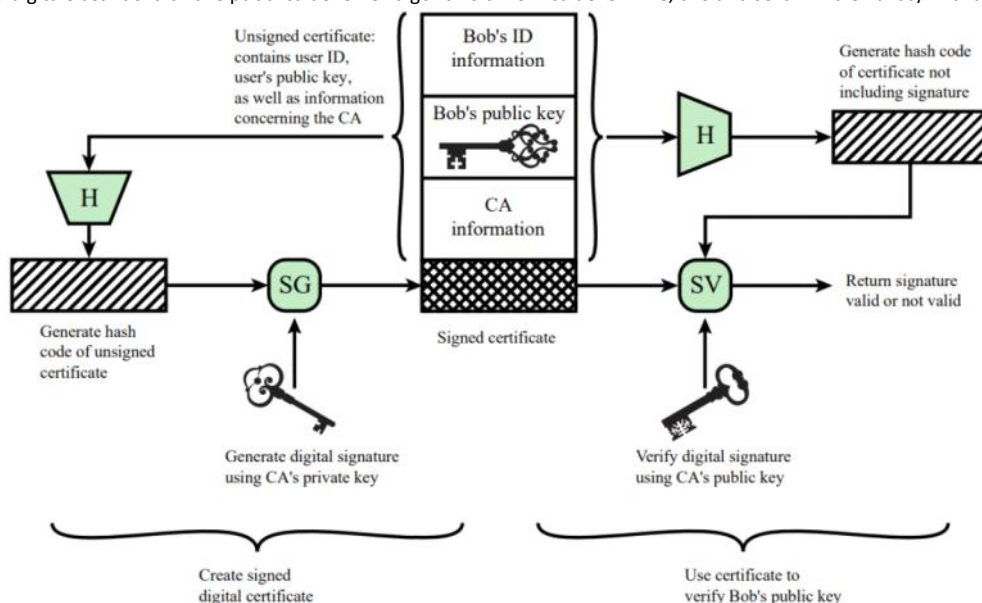
In questo modo un utente può pubblicare la sua chiave pubblica al CA e ottenere un certificato firmato e poi pubblicare quel certificato così che se a qualcuno serve la chiave pubblica di quell'utente può verificare la validità tramite la firma del CA collegata alla chiave.

Creazione e Verifica del Certificato Firmato

- 1) Il cliente crea una chiave pubblica e una chiave privata
- 2) Il cliente prepara un **certificato non firmato** che include l'User ID e la sua chiave pubblica e lo invia al CA in un modo sicuro
- 3) Il CA **crea la firma**:
 - a. Calcola l'hash del certificato non firmato
 - b. Crea una firma digitale usando la chiave privata del CA e un algoritmo di generazione delle firme

Ora l'utente può fornire il certificato firmato a ogni altro utente e ogni altro utente può verificare la validità del certificato in due modi:

- a) Calcolando l'hash del certificato (senza includere la firma)
- b) Verificando la firma digitale usando la chiave pubblica del CA e l'algoritmo di verifica delle firme, che dirà se la firma è valida/invalida



Buste Digitali (Digital Envelopes)

La crittografia asimmetrica è computazionalmente troppo lenta per cifrare grandi moli di dati. Per unire la velocità della crittografia simmetrica con la sicurezza della distribuzione asimmetrica si usano le **buste digitali (digital envelopes)**.

Creazione e Invio (Bob)

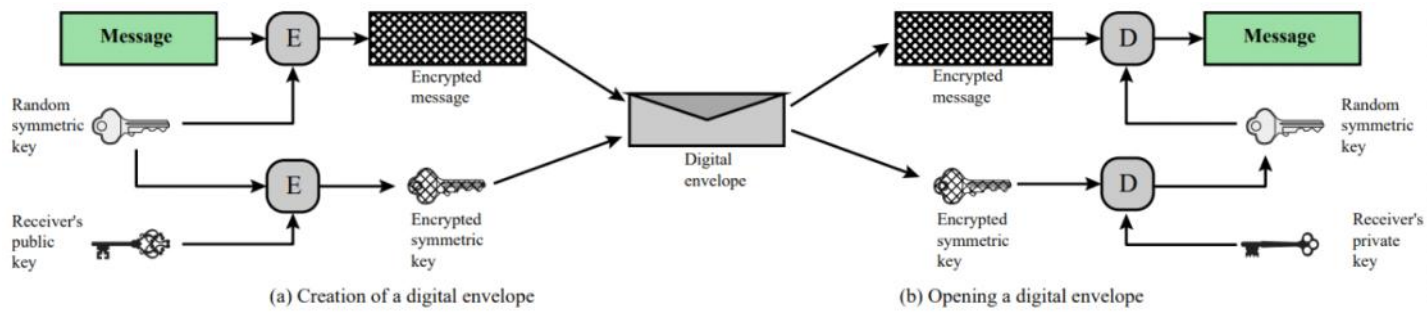
- Bob genera una **"session key"** (una chiave simmetrica randomica usa e getta)
- Usa la session key per cifrare il messaggio usando la crittografia simmetrica

Ricezione (Alice)

Solo Alice può decriptare la session key che userà poi per decriptare il messaggio originale.

- Usa la **chiave pubblica del destinatario** per cifrare la session key usando la crittografia asimmetrica
 - Aggiunge la session key cifrata al messaggio cifrato e lo invia ad Alice
- Quindi il messaggio è cifrato con la session key e la session key è cifrata con la chiave pubblica

Se Bob ottiene la chiave pubblica di Alice tramite il certificato della chiave pubblica allora Bob è sicuro che sia valida



Numeri Random

sabato 9 maggio 2026 19:12

I **numeri casuali** costituiscono lo scheletro di molti protocolli di sicurezza crittografica. Le loro applicazioni includono:

- Generazione di **chiavi** per **algoritmi a chiave pubblica** come RSA
- Creazione di **chiavi di flusso** (stream keys) per cifrari simmetrici continui
- Generazione di **session keys** temporanee per lo scambio tramite **digital envelopes**
- Impiego nei protocolli di **handshaking** per ostacolare i replay attacks

Requisiti di Casualità e Imprevedibilità

- 1) **Casualità (Randomness)**: la sequenza di numeri deve superare test statistici basati su due criteri:
 - **Distribuzione uniforme**: la frequenza di occorrenza di ciascun num. deve essere approssimativamente uguale
 - **Indipendenza**: nessun valore della sequenza può essere dedotto basandosi dagli altri valori (difficile da provare)
- 2) **Imprevedibilità (Unpredictability)**:
 - Ogni num. deve essere statisticamente indipendente da ogni altro numero della sequenza
 - Un avversario, pur conoscendo gli elementi precedenti della sequenza, non deve essere in grado di prevedere gli elementi futuri

Random Vs Pseudorandom Generators

Nei sistemi informatici esistono due approcci metodologici per la creazione di numeri random:

- **PRNG (Pseudorandom Number Generator)**: generatori che usano **algoritmi crittografici deterministici**, ed essendo deterministici non producono sequenze statisticamente random e sono teoricamente prevedibili se un attaccante riesce a scoprire il **seed iniziale** e l'algoritmo impiegato. Se l'algoritmo è fatto bene la sequenza soddisferà comunque i test statistici di casualità
- **TRNG (True Random Number Generator)**: generatori che non usano algoritmi matematici, ma campionano fonti hardware non deterministiche e processi naturali caotici e imprevedibili (es rumore termico dei circuiti elettronici, effetti quantistici, tempistiche irregolari di risposta dei dischi rigidi, ecc) I TRNG sono integrati in misura sempre maggiore all'interno dei microprocessori moderni per garantire livelli di entropia crittografica sempre migliore

Modalità di Funzionamento (Modes of Operation)

Per aumentare la sicurezza della crittografia simmetrica a blocchi, e superare le debolezze di ECB, sono state sviluppate tecniche alternative chiamate **modalità di funzionamento**. Esse sono:

- **Cipher Block Chaining (CBC):** ideato per superare le limitazioni dell'ECB, il CBC "incatena" l'output crittografico.
Prima di cifrare il blocco plaintext corrente, esso viene messo in **XOR col blocco ciphertext precedente**. Il primo blocco plaintext viene messo in XOR con un **blocco fittizio** chiamato **Initialization Vector (IV)**. In questo modo, pattern ripetuti nel testo originale produrranno blocchi cifrati sempre differenti.
Propagazione dell'errore in CBC: se si verifica un singolo errore di trasmissione (es bit flippato) nel blocco cifrato, l'errore si propaga nel calcolo XOR dei blocchi successivi durante la decifrazione, corrompendo il testo decifrato finale nei blocchi in cui il blocco corrotto viene usato come input nella decifrazione.
Es: se C_k è il blocco corrotto:
 - Per decifrare il blocco P_k usiamo il blocco corrotto C_k e il blocco cifrato precedente C_{k-1} , quindi P_k sarà corrotto
 - Per decifrare il blocco P_{k+1} usiamo il blocco buono C_{k+1} e il blocco cifrato precedente C_k (quello corrotto), quindi P_{k+1} sarà corrotto
 - Per decifrare il blocco P_{k+2} usiamo il blocco buono C_{k+2} e il blocco cifrato precedente C_{k+1} (buono), quindi l'output tornerà normale
- **Cipher Feedback (CFB):**

Algoritmi di Crittografia in Dettaglio

venerdì 8 maggio 2026 11:53